

Assignment 9: Feature Scaling and Model Evaluation

Q1: Feature Scaling Concepts

Instructions: Explain the following:

- Standardization
- Min-Max Scaling
- Robust scalling
- Why feature scaling is important

Scaling ensures that all your data uses the same "ruler" so the computer doesn't think a large number (like salary) is more important than a small number (like age). It prevents the model from being biased and helps it learn the patterns much faster.

- This method centers your data so that the average becomes zero and measures everything else by how many "steps" it sits away from that middle. It is great for data that follows a natural bell curve because it keeps the overall shape while making different features easier to compare.
- This method takes your smallest value and calls it "0," takes your largest and calls it "1," and fits everything else in between like a percentage bar. It is perfect for when you need fixed boundaries, but it can be easily ruined if you have one "giant" outlier that squashes all other data points.
- This focuses on the "typical" middle values of your data and ignores extreme outliers that would otherwise confuse the model. By using the median instead of the average, it stays reliable even if your dataset contains some weird or "noisy" numbers that don't belong.

Q2: Load and Explore Dataset

Instructions:

- Load the breast cancer dataset from sklearn.datas
- Display shape and first 5 rows

```
In [26]: from sklearn.datasets import load_breast_cancer
import pandas as pd
data_c = load_breast_cancer()
df = pd.DataFrame(data_c.data, columns=data_c.feature_names)
```

```
print(f'Shape of the df: {df.shape}')
print(f'First 5 rows of the df:\n{df.head()}')
```

Shape of the df: (569, 30)

First 5 rows of the df:

	mean radius	mean texture	...	worst symmetry	worst fractal dimension
0	17.99	10.38	...	0.4601	
0.11890					
1	20.57	17.77	...	0.2750	
0.08902					
2	19.69	21.25	...	0.3613	
0.08758					
3	11.42	20.38	...	0.6638	
0.17300					
4	20.29	14.34	...	0.2364	
0.07678					

[5 rows x 30 columns]

Q3: Train-Test Split

Split data into training and testing sets (80/20)

```
In [27]: from sklearn.model_selection import train_test_split

X = df
y = data_c.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Q4: Apply Feature Scaling

Apply:

- StandardScaler
- MinMaxScaler
- RobustScaler

```
In [28]: from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler

std_scaler = StandardScaler()
X_train_std = std_scaler.fit_transform(X_train)
X_test_std = std_scaler.transform(X_test)

min_max_scaler = MinMaxScaler()
X_train_min_max = min_max_scaler.fit_transform(X_train)
X_test_min_max = min_max_scaler.transform(X_test)

robust_scaler = RobustScaler()
X_train_rob = robust_scaler.fit_transform(X_train)
```

```
X_test_rob = robust_scaler.transform(X_test)
```

Q5: Train Models

Train Logistic Regression on:

1. Unscaled data
2. Standardized scaled data
3. MinMax scaled data
4. Robust Scaled data

```
In [29]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Unscaled data
model_un = LogisticRegression(max_iter=10000)
model_un.fit(X_train, y_train)
acc_un_model = accuracy_score(y_test, model_un.predict(X_test))
print("Unscaled data accuracy score = ", acc_un_model)

# Standardized scaled data
model_std = LogisticRegression()
model_std.fit(X_train_std, y_train)
acc_std_model = accuracy_score(y_test, model_std.predict(X_test_std))
print("Standardized scaled data accuracy score = ", acc_std_model)

# MinMax scaled data
model_min_max = LogisticRegression()
model_min_max.fit(X_train_min_max, y_train)
acc_min_max = accuracy_score(y_test, model_min_max.predict(X_test_min_max))
print("MinMax scaled data accuracy score = ", acc_min_max)

# Robust scaled data
model_rob = LogisticRegression()
model_rob.fit(X_train_rob, y_train)
acc_rob = accuracy_score(y_test, model_rob.predict(X_test_rob))
print("Robust scaled data accuracy score = ", acc_rob)
```

```
Unscaled data accuracy score = 0.956140350877193
Standardized scaled data accuracy score = 0.9736842105263158
MinMax scaled data accuracy score = 0.9824561403508771
Robust scaled data accuracy score = 0.9824561403508771
```

Q6: Evaluate Each Model

Evaluate each model using the corresponding test set and print the following:

- Accuracy
- MAE
- RMSE

```
In [30]: from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

models = [
    ("No Scaling", model_un, X_test),
    ("STD scaling", model_std, X_test_std),
    ("MinMax Scaling", model_min_max, X_test_min_max),
    ("Robust Scaling", model_rob, X_test_rob)
]

print(f"{'Scaler'} {'Accuracy'} {'MAE'} {'RMSE'}")
print('-' * 55)

for name, model, x_data in models:
    predictions = model.predict(x_data)

    acc = accuracy_score(y_test, predictions)
    mae = mean_absolute_error(y_test, predictions)
    rmse = np.sqrt(mean_squared_error(y_test, predictions))

    print(f"{name} | {acc:0.2f} | {mae:0.2f} | {rmse:0.2f}")
```

Scaler	Accuracy	MAE	RMSE
No Scaling	0.96	0.04	0.21
STD scaling	0.97	0.03	0.16
MinMax Scaling	0.98	0.02	0.13
Robust Scaling	0.98	0.02	0.13

Q8: Reflection

Discuss:

- Impact of scaling on performance
- When scaling is necessary

Feature scaling levels the playing field by ensuring that large-scale numbers don't unfairly dominate smaller ones, which leads to more accurate and unbiased model weights. This process significantly speeds up the training of models like Logistic Regression or Neural Networks by making the mathematical path to the best solution smoother and more direct. It is essential for algorithms that calculate distances, such as KNN or SVM, though it is usually not required for tree-based models like Random Forests. Ultimately, scaling turns "messy" data into a standardized format that computers can process with much greater efficiency and precision.